

* Functions

⇒ Function is a block of reusable code that perform a specific task.

Working

- It helps in breaking a large program into small manageable parts.

Why?

- Increase code reusability.
- Improve readability and modularity.
- Easier debugging and testing.

Types

- ① Built-in Functions → provided by Python - print(), input(), lower(), len()
- ② User defined functions → created by the programmer using def().
- ③ lambda functions → anonymous, single line functions created with lambda.

② User defined functions

Syntax

```
def fun-name (parameters):
```

```
    statements
```

```
    return value
```

• Different methods of UD functions

- 1) Function without input & without return.
- 2) Function with input & without return.
- 3) Function without input & with return.
- 4) Function with input & with return.

① Function without input & without return

Syntax

```
def (fun-name):
```

```
    statements
```

calling function

```
fun-name()
```

Ex.

```
def add():
```

```
    a = int(input("Enter the value"))
```

```
    b = int(input("Enter the value"))
```

```
    s = a + b
```

```
    print("Sum of a & b is", s)
```

```
add()
```


OLP

Enter the value : 34

Enter the value : 23

Sum of 34 & 23 is 57

Note^{xx}

- Local variable : Any variable declared inside the function is called local variable.
- Global variable : Any variable declared outside the function is called global variable.

(2)

Function with input & without return
Syntax

```
def fun_name(p1, p2, ..., pn):
    statements
```

calling functions

read the parameters

```
fun_name(p1, p2, ..., pn)
```

Ex:

```
def add2(n1, n2):
```

```
    m = n1 + n2
```

```
    print(f"sum of {n1} & {n2} is {m}")
```

```
add(5, 7)
```

OLP

Sum of 5 and 7 is 12

Ex 1: We are reuse the variable means we can write program outside

```
j = int(input("Enter the value"))  
k = int(input("Enter the value"))  
add2(j, k)
```

OLP Enter the value: 3
Enter the value: 4
Sum of 3 and 4 is 7

③ Function without input & with return
Syntax

```
def fun-name():  
    statements  
    return value
```

calling function

```
var = fun-name()
```

statements

Ex:

```
def add3():  
    a = int(input("Enter the value"))  
    b = int(input("Enter the value"))  
    s = a + b  
    return a, b, s
```


add3c)

OIP

Enter the value: 12

Enter the value: 67

(12, 67, 79)

x, y, z = add3c)

OIP

Enter the value: 34

Enter the value: 21

Print(&"sum of x & y is {z}")

OIP

Sum of 34 & 21 is 55

④

Function with input & with return
Syntax

def fun-name (p1, p2 ----, pn):

statements

return value

calling the function

read parameters

var = fun-name (p1, p2 ---, pn)

statements

Ex:

def add4 (n1, n2):

s = n1 + n2

return s

add4(3, 64)

O/P 67

ex 5: $x = 12$

$y = 24$

sum = add4(x, y)

print(f"sum of 12 & 24 is {sum}")

O/P sum of 12 & 24 is 36